Tim Pengajar IF2150

IF2150 – Rekayasa Perangkat Lunak

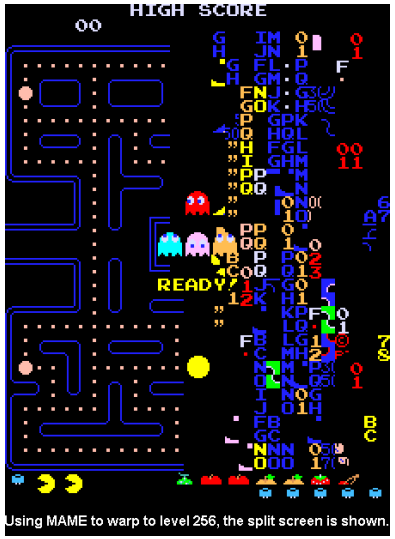
Software Testing

SEMESTER I TAHUN AJARAN 2025/2026

KNOWLEDGE & SOFTWARE ENGINEERING

Kenapa Perlu Pengujian

- Mariner I space probe (1962)
  - Suatu rumus yang ditulis dengan pensil, akhirnya salah diterjemahkan oleh pemrogram
- THERAC-25 radiation machine (1985) :
  - Pengujian kurang lengkap, hingga sempat menyebabkan 3 pasien meninggal
- Rudal Patriot (1991)
  - Salah sasaran karena kesalahan pembulatan (round off error)
- Roket Ariane 5 (1996)
  - Meledak karena kesalahan penanganan konversi komponen 16 bit ke 64 bit
- NASA's Mars lander: September 1999
  - Jatuh karena ada kesalahan dalam integrasi suatu unit
- Millenium Bug (Y2K Bug)
  - Penghilangan digit dua digit awal pada data tahun menjadi masalah ketika pergantian menuju tahun 2000
- Heartbleed (2014)
  - Kerentanan pada OpenSSL
- Video Game 'Pac Man' (2012)
  - Split screen di Level 256



Using MAME to warp to level 256, the split screen is shown.

- **http://en.wikipedia.org/wiki/List_of_software_bugs**
- https://www.computerworld.com/article/2515483/enterprise-applications/epic-failures--11-infamous-software-bugs.html

KNOWLEDGE & SOFTWARE ENGINEERING

Hungarian hacker arrested for pressing F12 (2017)

- The Budapest Transport Authority recently launched an online payment system with the help of a T-Systems Hungary, Deutsche Telekom's consulting arm.
  - The system, which took three months to buildThe software, not unexpectedly for such a project, was full of bugs including the discovery of an administration screen with with a password set to "adminadmin."
- On or about July 14 an 18-year-old – "The boy is nobody. He's not even a programmer," said one Hungarian who wished to remain anonymous – emailed BKK about a hole he found in their system. The hole, if it can be called that, let anyone with passing knowledge of modern browsers to set any price they wanted for any ticket in the system. By simply pressing F12 a "hacker" could change the price of a ticket right in the browser, and because there were no server checks, they could purchase the ticket at that price.

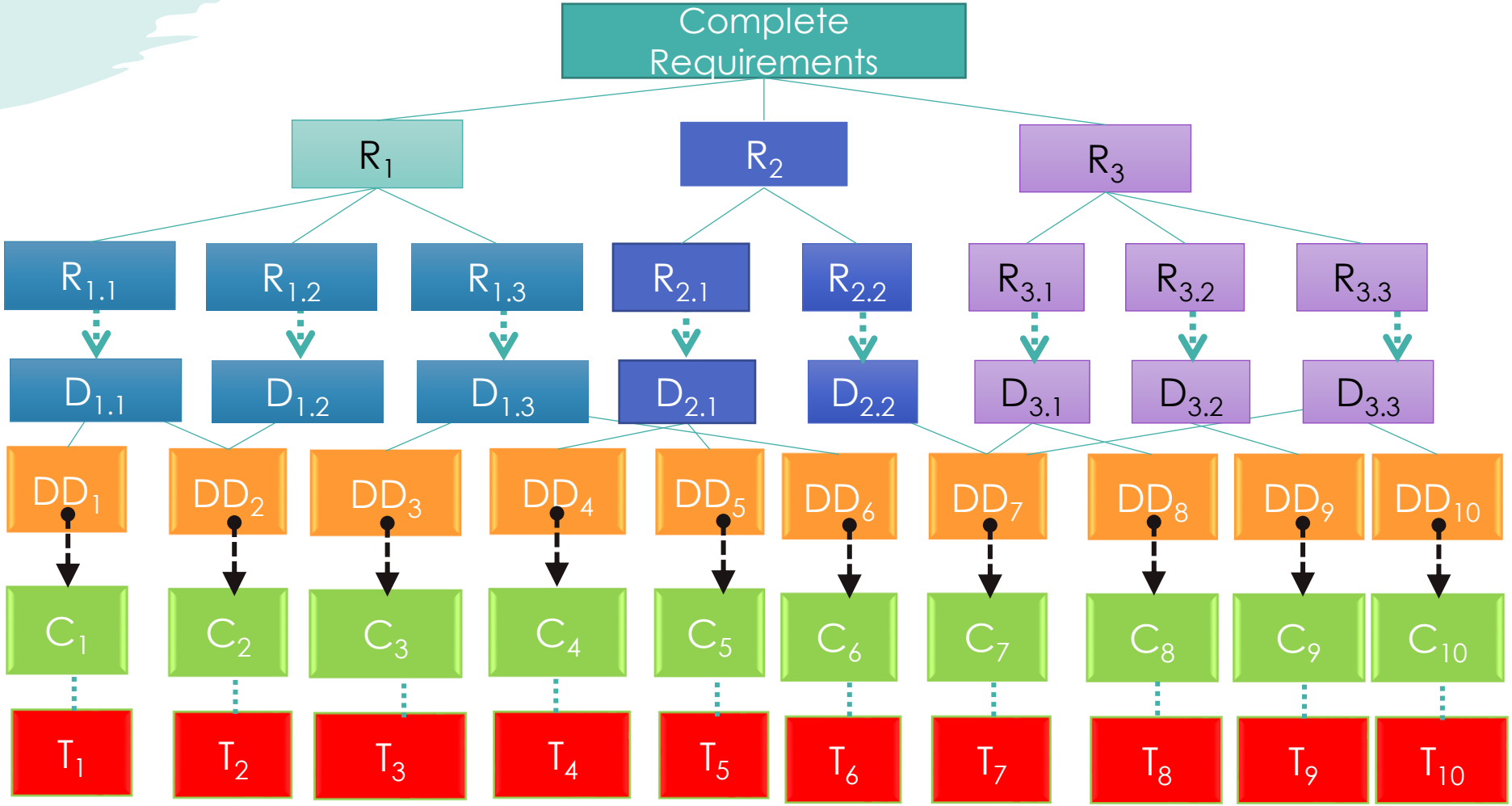
https://techcrunch.com/2017/07/25/hungarian-hacker-arrested-for-pressing-f12/

Pengujian Perangkat Lunak

Untuk "Penguji" (bukan pemrogram),
Pengujian adalah proses mencoba program dengan tujuan mencari kesalahan sebelum akhirnya diberikan ke pengguna

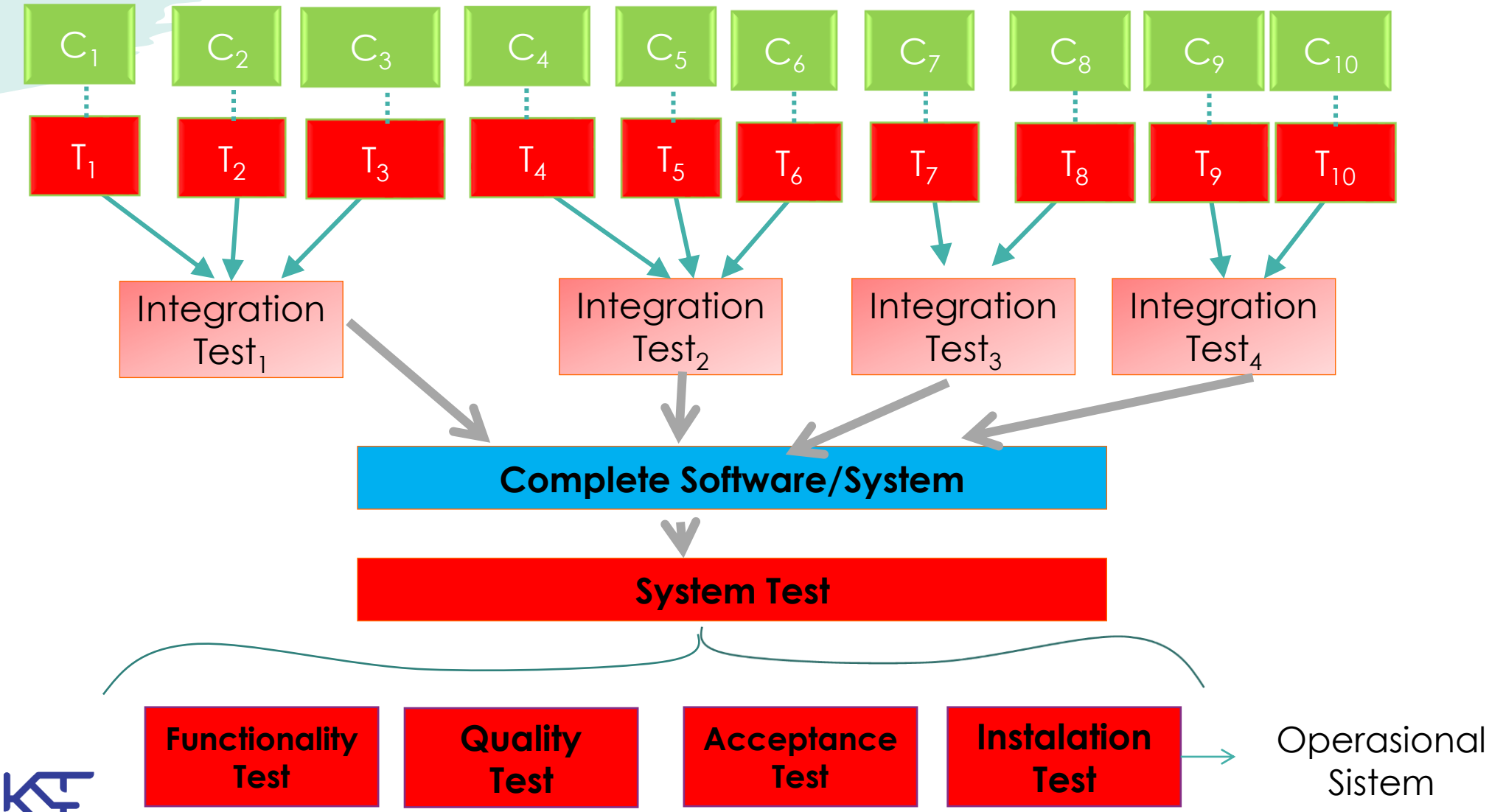
Untuk "Pemrogram" yang menguji program,
Pengujian adalah proses mencoba program dengan tujuan menunjukkan tidak ada salah pada programnya

Aktivitas Pengujian

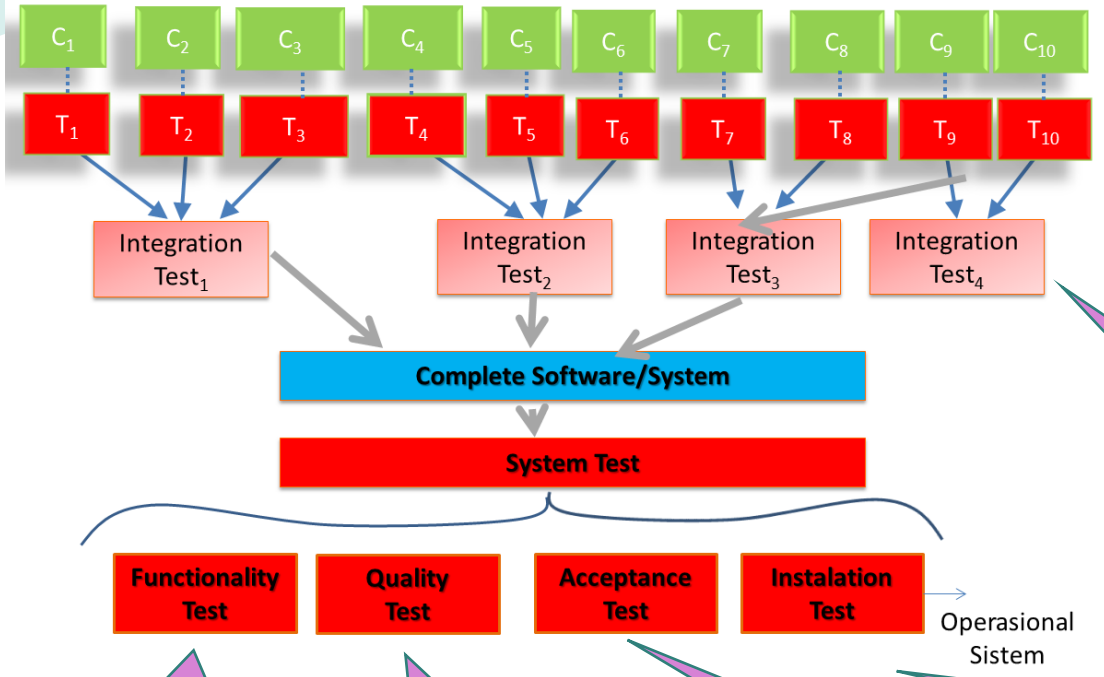


Pemrogram harus melakukan Pengujian terhadap kode programnya

Setiap unit program harus digabung dan hasil penggabungannya diuji kembali (Integration Testing)



Peran tiap pengujian



Memastikan tiap Komponen/Unit harus sesuai spesifikasi

Memastikan tiap Komponen/Unit harus dapat bekerjasama

Verifikasi tiap Kebutuhan Fungsional sudah terpenuhi

Verifikasi tiap Kebutuhan Non-Fungsional sudah terpenuhi

Verifikasi dari Calon Pengguna/Customer

Pengujian di lingkungan user

Pengujian memberikan indikasi:

Keberadaan errors

Ada perbedaan antara kebutuhan dengan implementasi

Performansi sistem

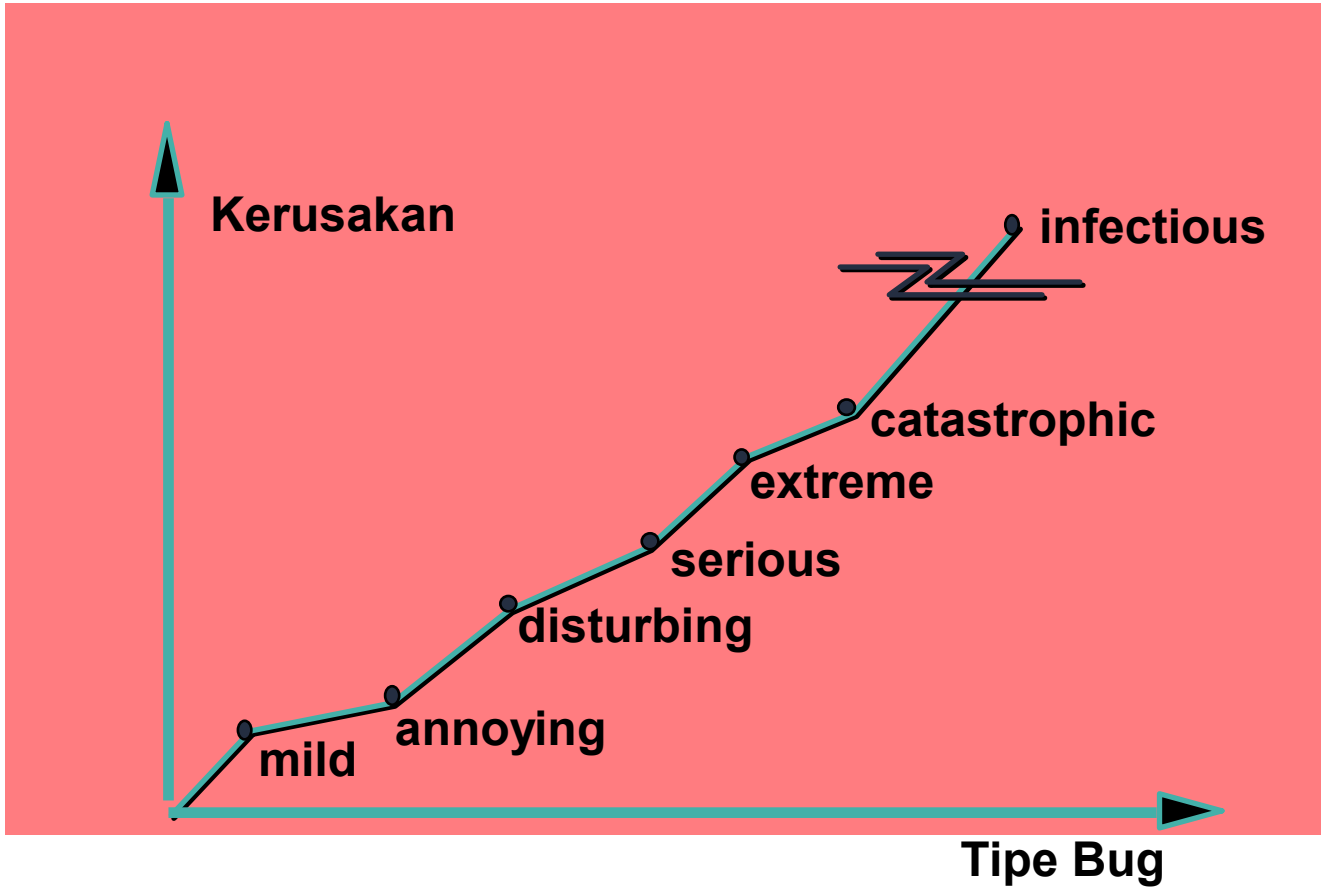
Kualitas Produk

Pengujian tidak harus menunjukkan sumber kesalahan, karena sumber kesalahan bisa saja berasal dari kode program, salah logika, salah desain, salah spesifikasi, dll

Failure, Error, Fault and Defect

- ***Failure***
  - Kegagalan (*failure*) terjadi kalau ada perilaku dari sistem yang tidak sesuai dengan permintaan di spesifikasi sistem
- ***Error***
  - Error adalah status dari sistem
  - Status error ini bisa menyebabkan kegagalan jika tidak ada perbaikan
- ***Fault***
  - Fault adalah sumber dari error
- ***Defect***
  - Sinonim dengan **Fault**
  - Sering disebut juga **bug**

Akibat dari Bug



Kategori Bug

- Bug di Fungsi
- Bug di Sistem
- Data Bugs
- Coding Bugs
- Design Bugs
- Documentation Bugs
- Standard Violations

Testing vs. Debugging

- Pengujian (testing) berbeda dengan debugging
  - Kasus uji yang sukses adalah kasus uji yang dapat menunjukkan bahwa ada salah dalam program
    - Kasus uji ini digunakan dalam pengujian
  - Debugging dilakukan bila sudah ditemukan suatu kesalahan, dan tujuannya mencari sumber kesalahan (fault atau defect)
    - 2 Langkah yang dilakukan:
      - Cari kesalahan
      - Perbaiki kesalahan
  - Debugging adalah menjadi bagian dari strategi pengujian

Strategi Debugging (1)

- Tujuan Debugging:
  - Mencari dan membenarkan penyebab terjadinya error
- Bugs dapat ditemukan dengan cara:
  - Evaluasi yang sistematis
  - Intuisi
  - 'Luck'
- Tiga strategi yang biasanya digunakan:
  - Brute Force
  - Backtrack
  - Cause Elimination

Myers, Glenford J., Corey Sandler, and Tom Badgett. *The art of software testing*. John Wiley & Sons, 2011.

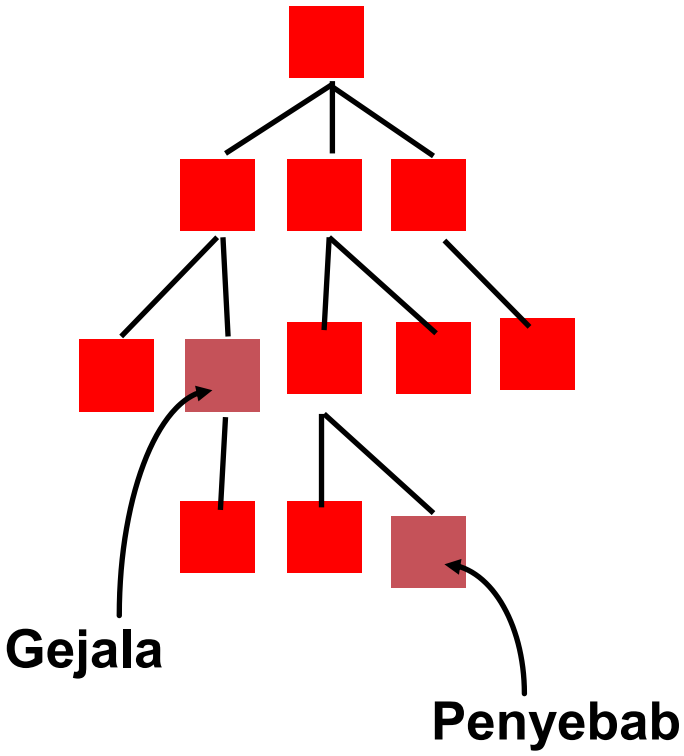
Strategi Debugging (2)

- **Brute force**
  - Teknik ini sering digunakan, dan paling tidak efisien dibandingkan yang lain
  - Biasanya digunakan bila cara lain sudah gagal
  - Melibatkan:
    - Memory dumps
    - Run-time trace
    - Output Statements
  - Sering banyak usaha dan waktu yang terbuang
- **Backtracking**
  - Lebih banyak berhasil dilakukan pada program kecil
  - *Source code* dibaca dimulai dari lokasi di mana symptom pertama kali ditemukan
  - Penelusuran manual dilakukan secara mundur (trace backward) sampai lokasi penyebab error ditemukan
  - Pada program besar, jumlah jalur mundur bisa sangat besar, sehingga metode ini menjadi tidak efisien

Strategi Debugging (3)

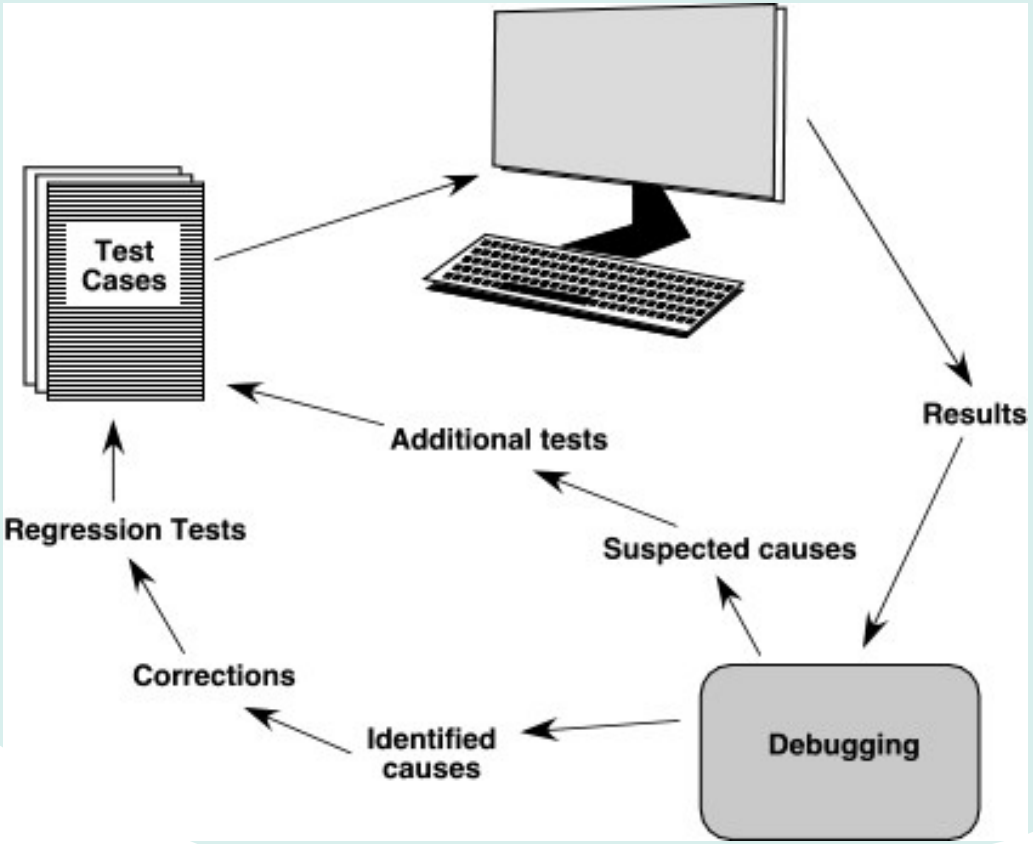
- **Cause elimination**
  - Menggunakan **Induksi** dan **Deduksi**
  - Induksi – dari detail ke lebih umum
    - Buktikan bahwa suatu kasus spesifik adalah benar, kemudian digunakan untuk membuktikan kasus umum adalah benar
  - Deduksi – dari umum ke detail
    - Dari sekumpulan premis umum digunakan sebagai konklusi yang lebih spesifik
  - Data yang terkait dengan kemunculan error diatur untuk mengisolasi potensi penyebab terjadinya kesalahan
    - Penyebab terjadinya error dibuat hipotesa, dan data-data tadi dapat digunakan untuk membuktikan atau sebaliknya
    - Alternatif lain, dibuat sekumpulan daftar penyebab yang mungkin dan pengujian dilakukan berdasarkan daftar tadi
    - Jika salah satu pengujian menunjukkan ada suatu penyebab yang khusus, kumpulan data diperkecil untuk mengisolasi bug

Symptoms & Causes



- Gejala dan penyebabnya mungkin terpisah
- Gejala mungkin hilang kalau suatu problem diperbaiki
- Penyebabnya mungkin karena kombinasi yang bukan error
- Penyebabnya mungkin karena sistem/*compiler error*
- Penyebabnya mungkin karena asumsi yang dipercayai benar
- Gejala mungkin muncul mungkin hilang

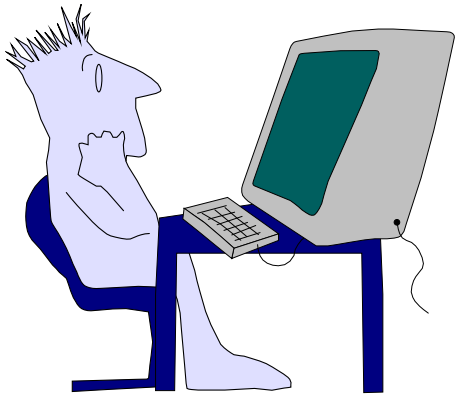
Proses Debugging



Memperbaiki 'Error'

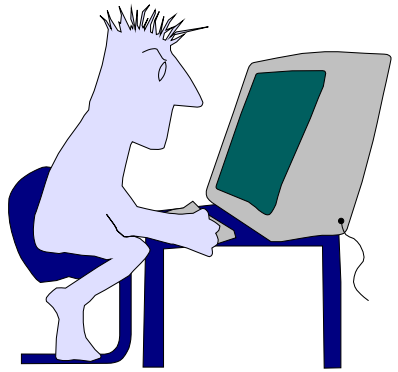
- Apakah penyebab suatu bug bisa direproduksi di bagian program lain
  - Pada banyak situasi, cacat dari suatu program disebabkan oleh adanya suatu pola error yang dari suatu bagian di tempat lain
- Apa bug berikutnya yang mungkin muncul setelah suatu perbaikan (bug fix) dilakukan?
  - Sebelum perbaikan dilakukan maka source code (atau bahkan desain) harus dievaluasi terhadap adanya coupling logik atau struktur data
- Apa yang dapat dilakukan agar suatu bug tidak terjadi sejak awal?
  - Jika 'software proses diperbaiki, maka bug akan otomatis terhindari dari program awal dan juga program-program berikutnya.

Siapa yang melakukan pengujian?



Pengembang

Mengerti sistem, tapi akan menguji untuk membuktikan bahwa program yang sudah dibuatnya sudah benar



Penguji Independen

Harus belajar sistemnya, tapi akan menguji untuk mencari bahwa program masih ada yang salah

Penguji independen: Client, User, third party

Aktivitas Pengujian

- Perencanaan Pengujian (Test Plan)
- Perancangan Pengujian (Test Design)
- Eksekusi Pengujian (Test Execution)
- Laporan Pengujian (Test Report)

Aktivitas Perencanaan Pengujian

- Perencanaan Pengujian
- Perancangan Pengujian
- Eksekusi Pengujian
- Laporan Pengujian

- Perencanaan apa saja yang akan diuji
- Perencanaan waktu
- Perencanaan sumberdaya yang diperlukan
  - Orang
  - Perangkat lunak pendukung
  - Alat/perangkat keras
  - Lingkungan pengujian
  - dan lain-lain
- Perencanaan prosedur umum pengujian

Aktivitas Perancangan Pengujian

- Perencanaan Pengujian
- Perancangan Pengujian
- Eksekusi Pengujian
- Laporan Pengujian

- Untuk apapun yang akan diuji, maka perlu dirancang Kasus Pengujian atau Kasus Uji (Test Case)

- Test Case berisi:
  - Masukan yang diuji
  - Harapan Pengujian
  - Cara menilai hasil eksekusi pengujian

Kasus Uji

Ada masukan data

Ada harapan hasil (expected result/outcome)

Masukan	Harapan Hasil
A=0	return 0
A=1	return 1
A=2	return 4

Contoh Pengujian untuk Fungsi Kuadrat (n)

Masukan	Harapan Hasil
n = 0	0
n = 1	1
n = 2	4
n = 3	9
n = 4	16
dst…	

Pertanyaan: Seberapa banyak jumlah kasus uji nya?

Kasus Uji yang Baik

- Memiliki kemungkinan tinggi untuk menemukan error
- Tidak redundan (duplikasi yang tidak perlu)
- Pilih teknik yang terbaik
  - Atau tepat sesuai dengan karakteristik software yang diuji
    - Web-based Application vs Mobile-based application
    - Aplikasi Perbankan vs. Aplikasi Perpustakaan
    - Aplikasi Saintifik vs. Aplikasi Sistem Informasi
    - Aplikasi Embedded system vs. Aplikasi Logistik
- Tidak terlalu sederhana juga tidak terlalu kompleks
- Kasus Uji yang sukses adalah jika kasus uji itu dapat membuktikan adanya error!

Aktivitas Eksekusi Pengujian

- Perencanaan Pengujian
- Perancangan Pengujian
- Eksekusi Pengujian
- Laporan Pengujian

- Dilakukan sesudah kode program selesai dikerjakan.
- Eksekusi program dilakukan dengan menggunakan masukan data yang sudah di rancang
- Jika harapan hasil tidak sesuai dengan hasil eksekusi, berarti perlu ada perbaikan program!

KNOWLEDGE & SOFTWARE ENGINEERING

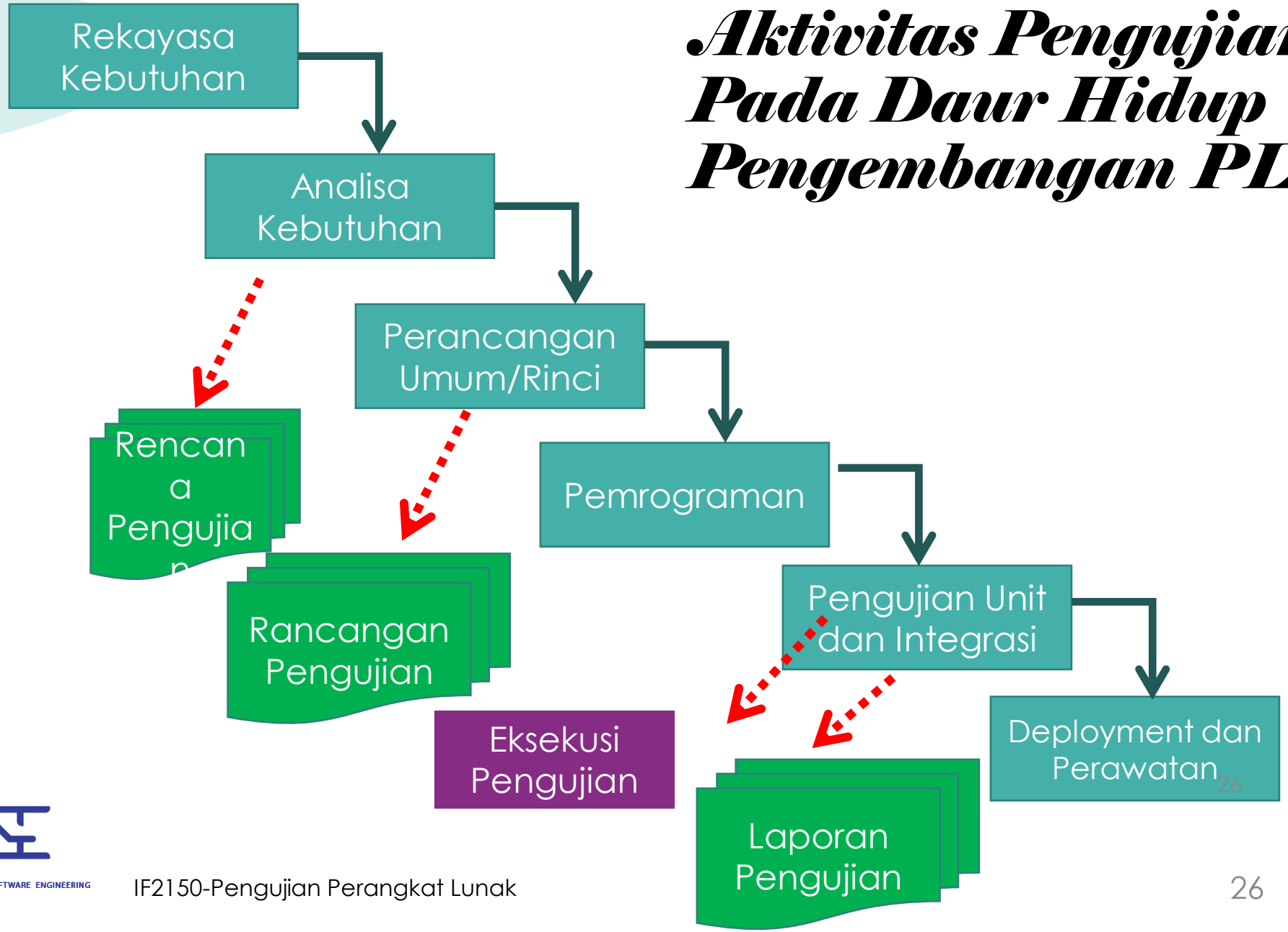
Aktivitas Pelaporan Hasil Pengujian

- Perencanaan Pengujian
- Perancangan Pengujian
- Eksekusi Pengujian
- Laporan Pengujian

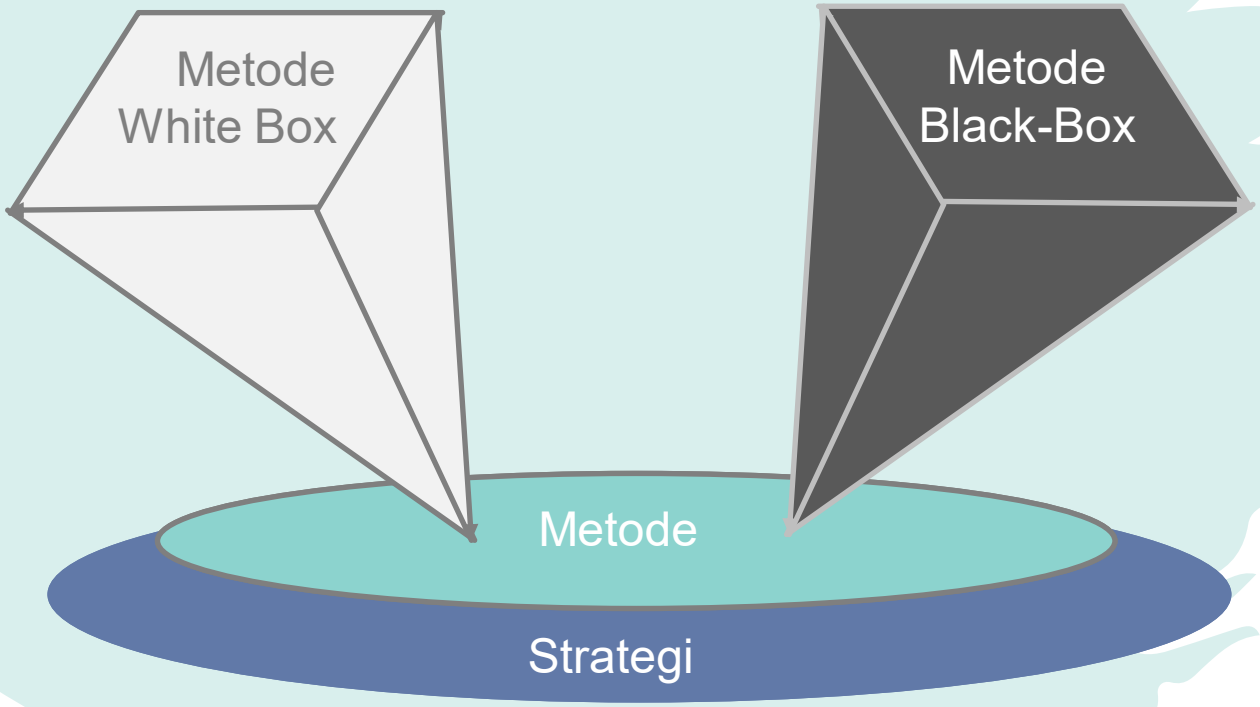
Laporan pengujian berisi laporan untuk setiap hasil eksekusi Test Case:

- Lulus pengujian jika hasil eksekusi program memberikan hasil yang sama dengan harapan hasil
- Tidak lulus pengujian jika hasil eksekusi memberikan hasil yang berbeda
- Kadang dapat disertai dengan catatan khusus

Aktivitas Pengujian Pada Daur Hidup Pengembangan PL



Metode/Strategi Pengujian



White Box Testing

Penguji bisa membaca kode program(bisa melihat bagian pengulangan, statement kondisi, pemanggilan prosedur/fungsi, dan lain-lain), sehingga kasus uji dibuat berdasarkan informasi dari kode program ini

Black Box Testing

Penguji tidak membaca source-code, sehingga pengembangan kasus uji berdasarkan spesifikasi fungsional dari program. Sering disebut juga sebagai pengujian fungsional

V & V

- *Verifikasi*
  - Aktivitas untuk menjamin bahwa **proses implementasi** suatu fungsi sudah di implementasikan dengan cara yang benar
- *Validasi*
  - Sekumpulan aktivitas untuk menjamin bahwa **fungsi sudah dibuat dengan benar** sesuai dengan kebutuhan pengguna
    - Boehm [Boe81]:
      - *Verification:* "Are we building the product right?"
      - *Validation:* "Are we building the right product?"

V vs. V

	Verification	Validation
Tujuan	Mengevaluasi pengembangan produk sesuai dengan spesifikasi kebutuhan dan perancangan	Mengevaluasi produk terhadap kebutuhan/harapan pengguna
Aktivitas	Review , Meeting, Inspeksi	Black box/white box testing
Pelaku	Tim QA (Quality Assurance)	Tim Penguji (Testing Tim)
Eksekusi Kode	Tidak ada eksekusi kode program, hanya memeriksa dokumen, model	Perlu ada eksekusi kode program
Biaya perbaikan kesalahan	Biaya lebih murah, kesalahan dapat ditemukan pada tahapan lebih awal	Biaya lebih mahal Karena kesalahan ditemukan di akhir

Strategi Pengujian

- Dimulai dari yang kecil ('testing-in-the-small') hingga bagian yang lebih besar ('testing-in-the-large')
- Untuk software biasa
  - Fokus awal adalah pengujian modul (unit)
  - Diikuti dengan pengujian integrasi modul
- Untuk software OO
  - Fokusnya pada pengujian pada setiap kelas yang melibatkan atribut dan operasinya
  - Berikut komunikasi/kolaborasi antar kelas

Pengujian yang efektif

- Lakukan **Technical Review**
  - Banyak ‘error’ akan ditemukan sebelum pengujian benar-benar dilakukan
- Pengujian dimulai dari elemen yang paling dasar hingga integrasi antar modul dan integrasi dengan sistem
- Teknik pengujian yang dipilih disesuaikan dengan teknik pendekatan software engineering
  - Konvensional? OO ?
- Pengujian dilakukan oleh developer
  - Untuk skala besar dilakukan oleh kelompok pengujian yang independen dari pengembang

Pengujian Unit atau Pengujian Modul atau Pengujian Komponen

(UNIT TESTING, MODULE TESTING OR COMPONENT TESTING)

Proses Pengujian

```c
#include <stdio.h>

int Kuadrat(int A)
{
    int hasil;
    hasil = 0;
    for (int i = 0; i < A;  i++)
    {
        hasil = hasil + A;
    }
    return hasil;
}
```

Unit Program yang akan diuji

Pembuatan Kasus Uji

Masukan	Harapan Hasil
A=0	return 0
A=1	return 1
A=2	return 4
A=3	return 9
A=10	return 100

Eksekusi Kasus Uji

Hasil:
- Kasus uji no x tidak lolos; atau
- Semua kasus uji lolos;

Perbaiki Kode

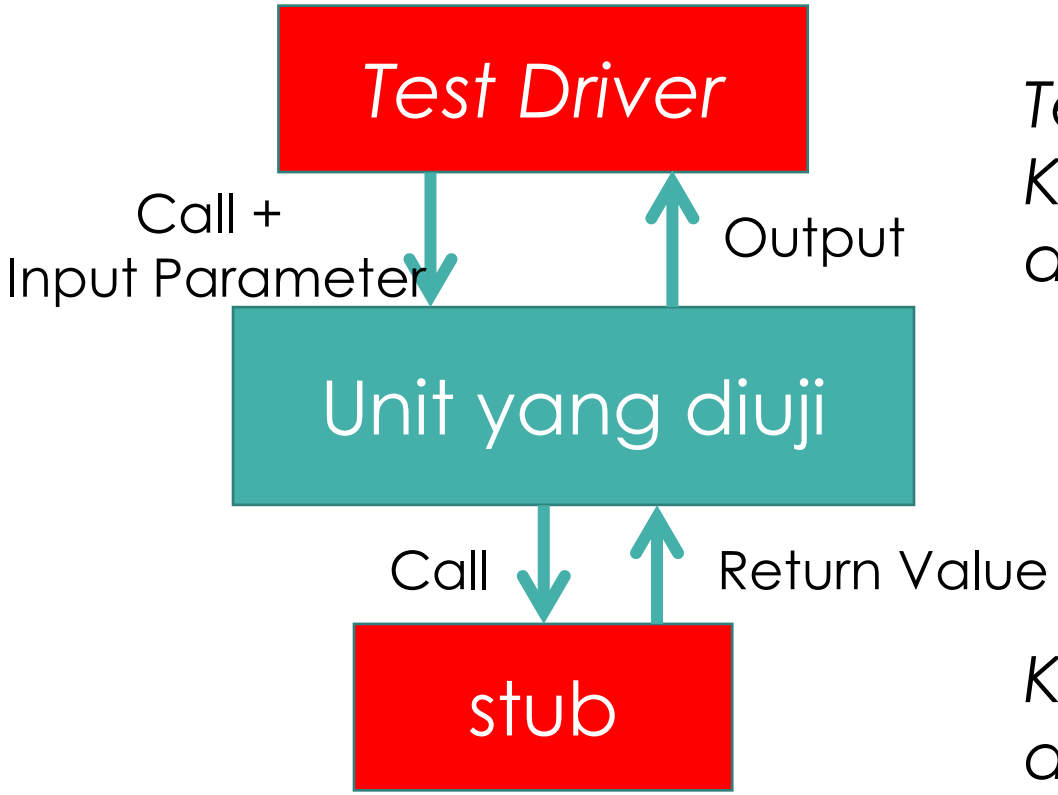
Selesai

Lingkungan Pengujian

- Lingkungan untuk menguji harus disiapkan
  - Perlu computer, perlu compiler, perlu persiapan jaringan, dan lain-lain
- Untuk pengujian Unit, maka mungkin saja ada bagian program lain yang belum siap, tetapi pengujian unit tetap harus dilakukan
  - Sehingga perlu disiapkan **TEST DRIVER** dan **STUB**
- *Test driver* adalah program yang memanggil unit yang sedang diuji, sekaligus memberikan input data, dan memeriksa hasilnya
- Bila unit yang akan diuji, perlu memanggil unit lain yang belum selesai di program, maka dipersiapkan STUB untuk meng-emulasi peran unit lain tadi.
  - Bagian ini disebut juga sebagai program dummy

- Test driver dan stub secara bersama-sama disebut **scaffolding**

Lingkungan Pengujian Unit



Test Driver memanggil Kode program yang diuji

Kode Program yang diuji memanggil Stub

Unit Under Test

```
int HitungGajiPegawai(int NIP)
{
    int GaPok = ReadGajiPokok(NIP);
    int GaTun = ReadGajiTun( NIP);

    return GaPok + GaTun;
```

STUB

```
int ReadGajiPokok(int NIP)
{
    // execute sql statement
    // select gaji from Pegawai
    return 2500;
}
```

```
int ReadGajiTun(int NIP)
{
    execute sql statement
    select tunj from Pegawai
    return 75000;
```

Test Driver

```
int main()
{
    assert(HitungGajiPegawai(111), 10000);
    assert(HitungGajiPegawai(200), 5000);
}
```

Unit Under Test

```
int HitungGajiPegawai(int NIP)
{
    int GaPok = ReadGajiPokok(NIP);
    int GaTun = ReadGajiTun( NIP);

    return GaPok + GaTun;
```

- Procedure HitungGajiPegawai yang ditulis dalam bahasa C ini adalah bagian yang diberikan oleh user untuk diuji oleh penguji
- Dalam pengembangan software skala menengah, pengujian unit ini dapat dilakukan oleh pengembang sendiri. Hal ini dilakukan untuk meminimalisasi biaya. Tentunya, pengembang biasanya menggunakan alat bantu seperti JUnit (java), NUnit (.NET), atau PUnit (PHP).

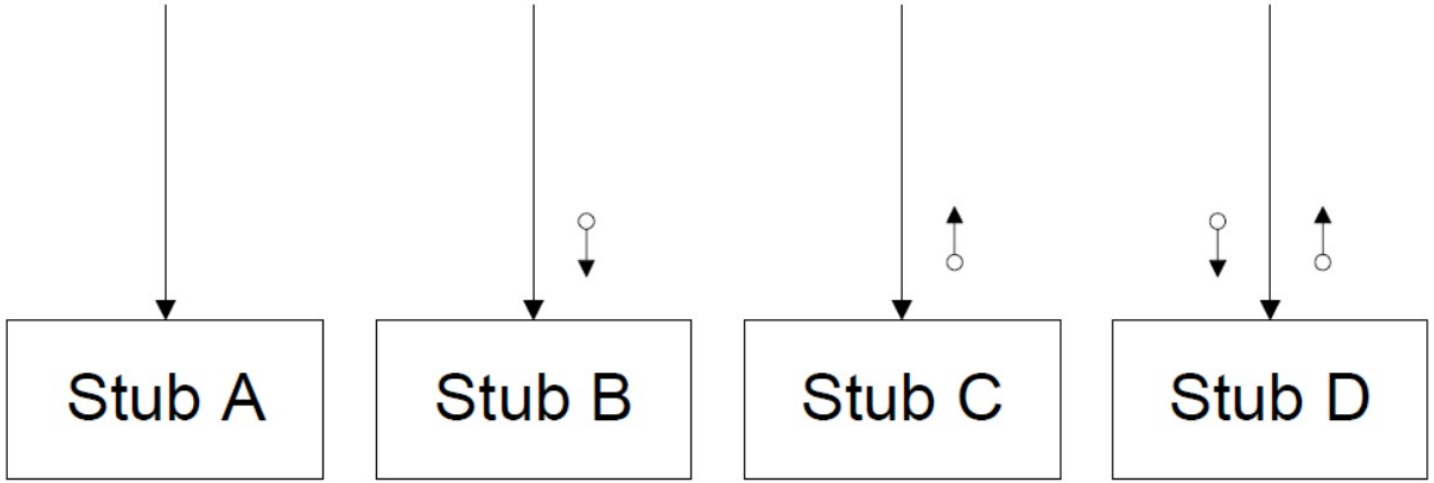
- Stub dibuat oleh penguji
  - Jika pengujian unit dilakukan oleh programmer, maka stub dibuat oleh programmer
- Stub dibuat karena bagian program itu belum siap atau belum selesai dikerjakan
  - Biasanya Stub dibuat untuk bagian yang ber-interface ke sistem lain.
    - Pada contoh ini, kedua stub seharusnya memanggil suatu perintah SQL ke basisdata, tetapi pada saat itu, pengembangan basisdata masih belum siap/selesai.
- Stub dibuat agar memungkinkan program yang diuji dapat di-compile dan melakukan fungsi yang diharapkan
  - Sehingga menghindari ketergantungan pada modul/unit lain yang belum selesai

STUB

```
int ReadGajiPokok(int NIP)
{
    // execute sql statement
    // select gaji from Pegawai
    return 2500;
}
```

```
int ReadGajiTun(int NIP)
{
    // execute sql statement
    // select tunj from Pegawai
    // return 75000;
}
```

Jenis STUB



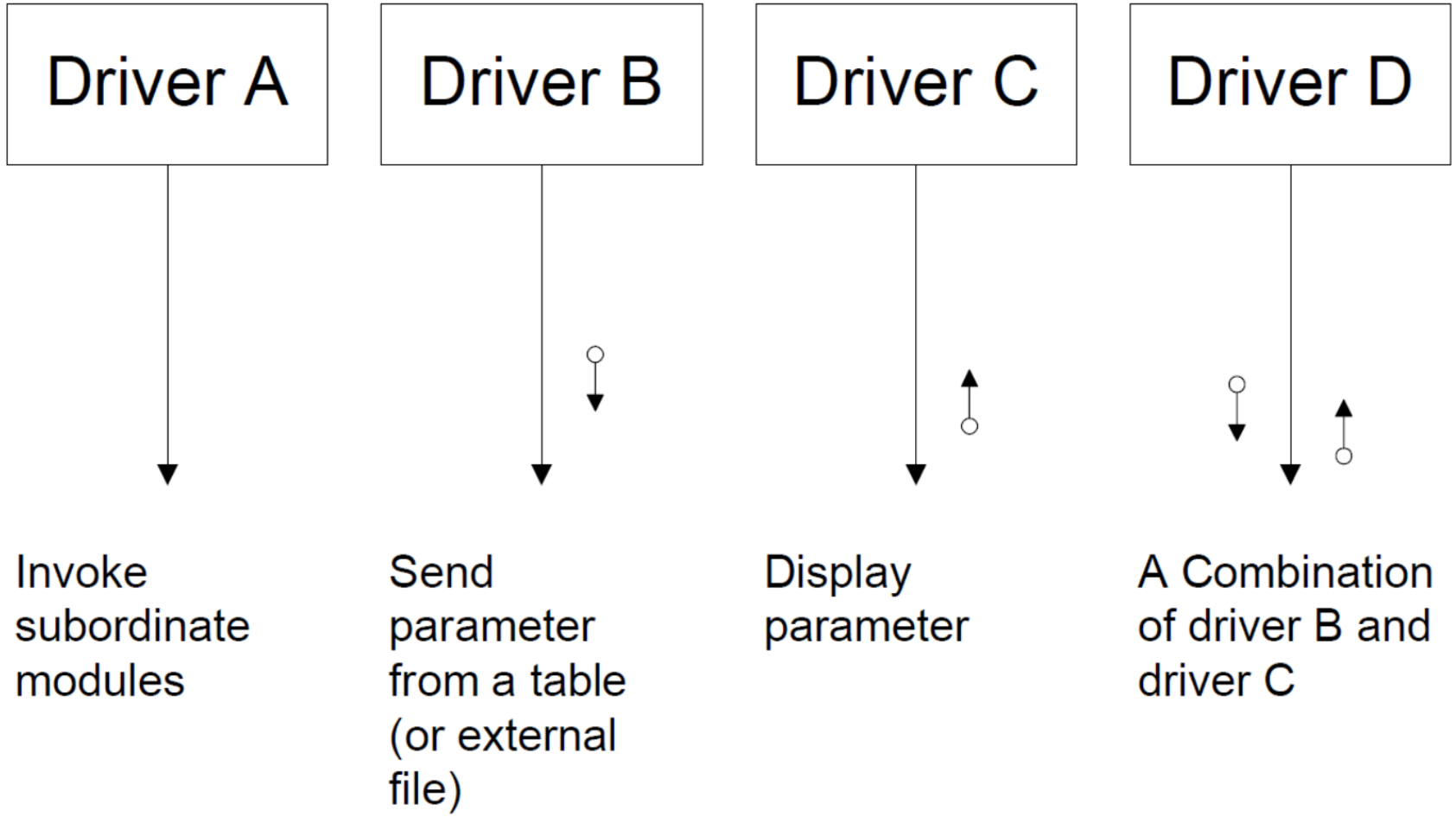
Stub A	Stub B	Stub C	Stub D
Display a trace message	Display passed parameter	Return a value from a table (or external file)	Do a table search for input parameter and return associated output parameter

- Test driver dikembangkan oleh penguji
  - Dan kalau program cukup kecil, maka programmer bertanggung jawab untuk mengembangkan test driver ini
- Test driver akan memanggil unit yang akan diuji
  - Test driver ini biasanya berisi beberapa kasus uji yang terkait dengan modul yang akan diuji
  - Test driver juga bisa berisi urutan skenario yang menggunakan unit yang diuji
- Test driver juga perlu memperhatikan penggunaan atau penyiapan stub
  - test driver menyesuaikan pemakaian stub sesuai kebutuhan, atau
  - Stub menyesuaikan dengan kebutuhan test driver
- Pada contoh dibawah ini, test driver menggunakan perintah 'assert' untuk melihat apakah fungsi HitungGajiPegawai mengembalikan suatu nilai
  - Dengan parameter 111, diharapkan mengembalikan nilai 10000

Test Driver

```
int main()
{
   assert(HitungGajiPegawai(111), 10000);
   assert(HitungGajiPegawai(200), 5000);
}
```

Jenis Driver



WHITE BOX TESTING

CONTOH:

- CONTROL FLOW TESTING
- DATA FLOW TESTING
- MUTATION TESTING
- DAN LAIN-LAIN

Control Flow Testing

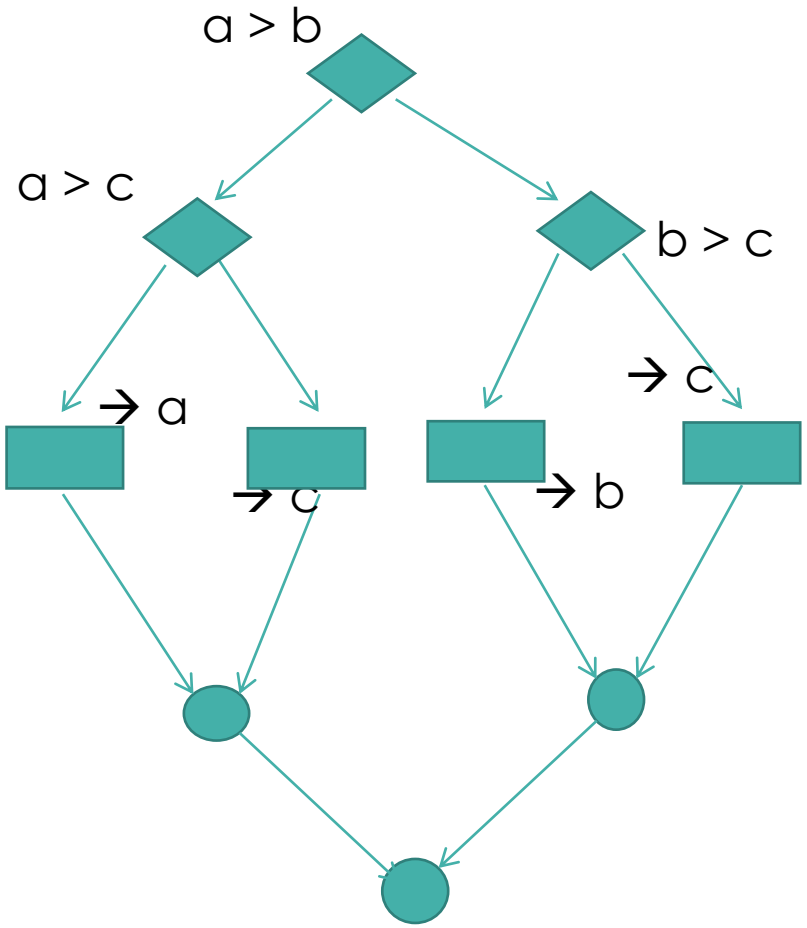
- Langkah
  - Kode Program diubah menjadi Control Flow Graph
  - Dilihat jalur eksekusinya
  - Setiap jalur eksekusi harus dilewati oleh kasus uji

Contoh Mencari Bilangan Terbesar

```
Function Largest( a, b, c : integer): integer
Kamus:-
Algoritma

    if a > b then
        if a > c then
            return a
        else
            return c
    else
        if b > c then
            return b
        else
            return c
```

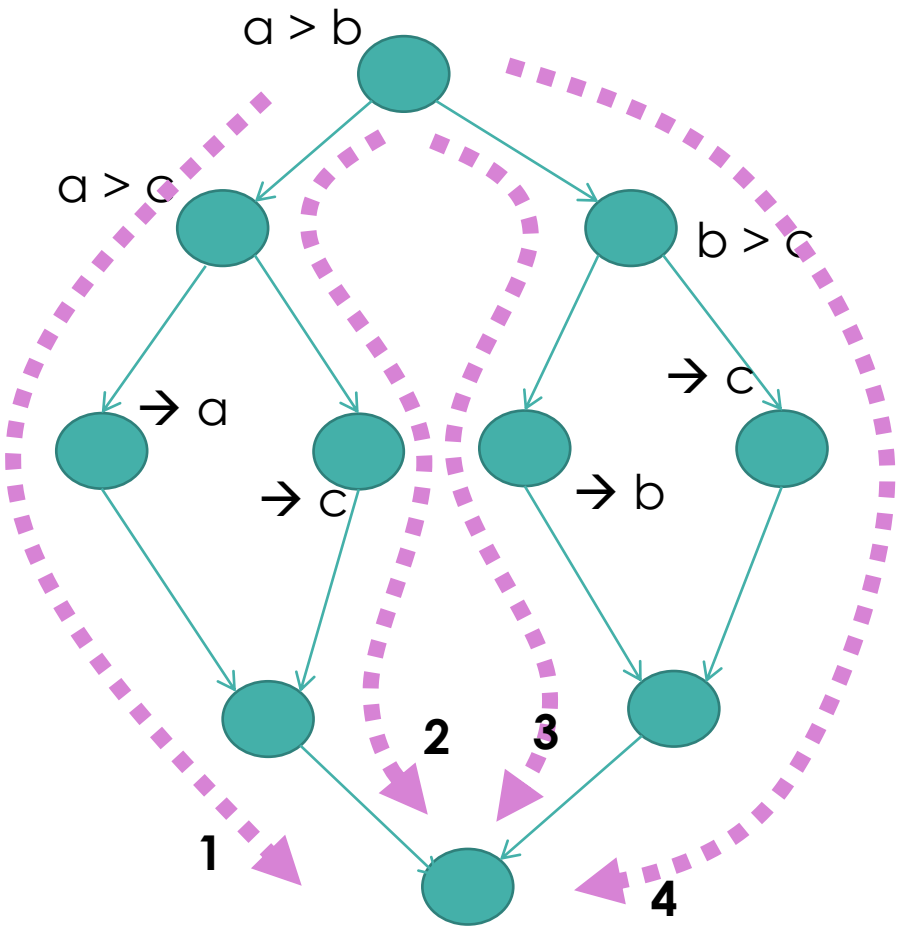
CFG

```
if a > b then
    if a > c then
        return a
    else
        return c
else
    if b > c then
        return b
    else
        return c
```



CFG dapat disederhanakan menjadi kumpulan nodes saja

```
if a > b then
    if a > c then
        return a
    else
        return c
else
    if b > c then
        return b
    else
        return c
```



Ada berapa jalur dasar?

Cari semua masukan yang memungkinkan semua jalur di lewati

Jalur	Input (a, b, c)	Hasil yang diharapkan
Jalur 1	(10, 3, 2)	10
Jalur 2	(25, 15, 40)	25
Jalur 3	(10, 15, 6)	15
Jalur 4	(10, 15, 19)	19

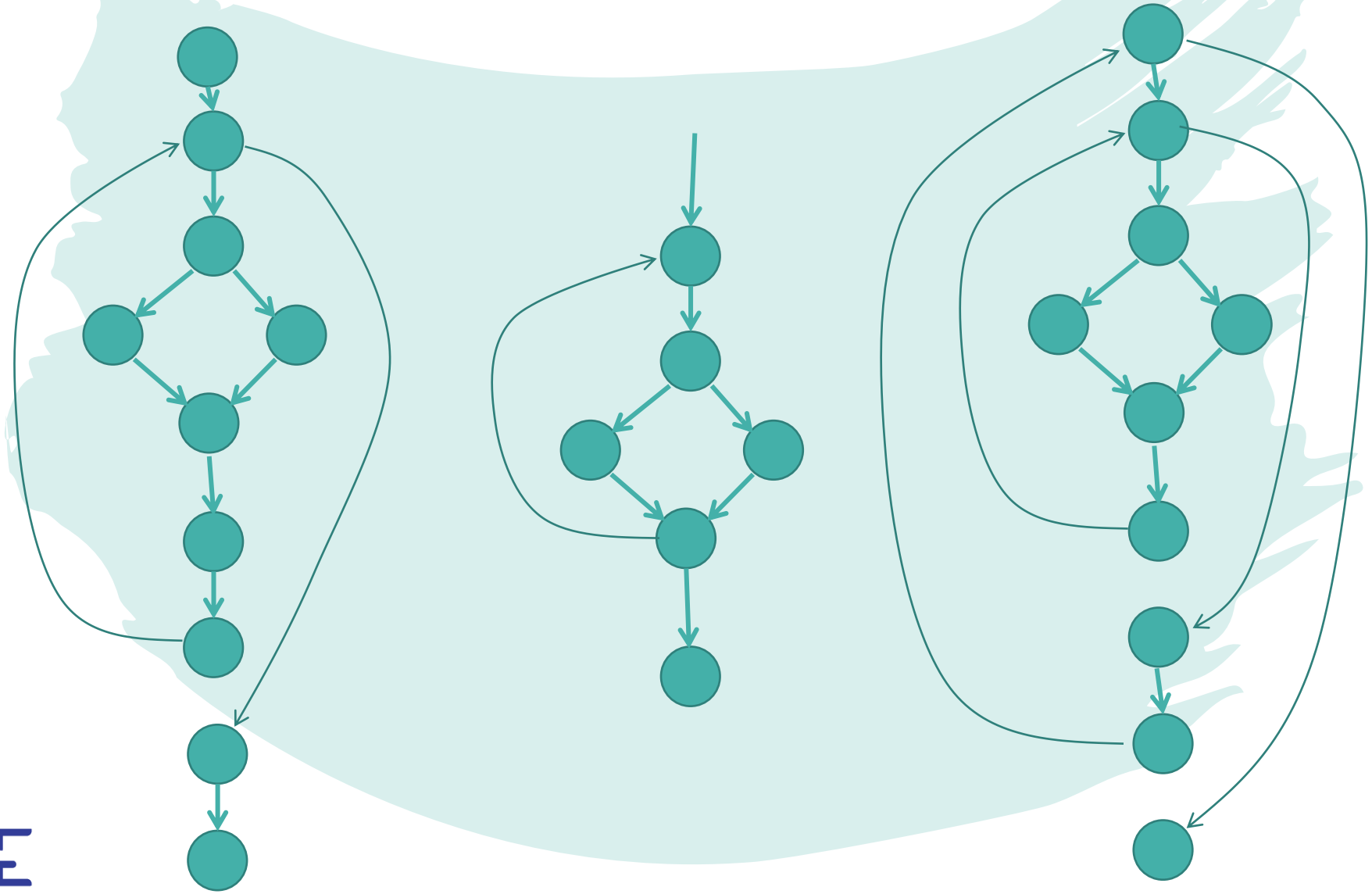
Buat CFG!
Dari C code berikut

```
#include <stdio.h>


int main()
{
    int x;
    a = 5;
    x = 0;
    while (x < 10)
    {
            printf( "%d %d\n", x, a );
        x = x + 1;
    }
}
```

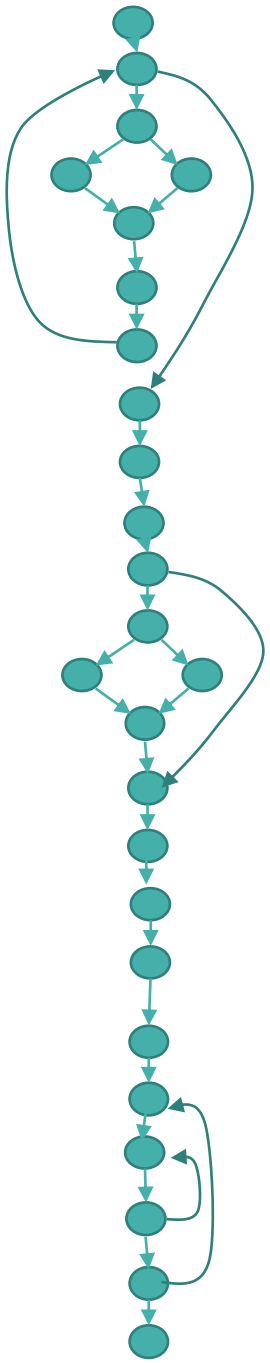
```
#include <stdio.h>

int main()
{
    int x;
    X = 0;
    do {
          if (x < 1)
             printf(“%d”, x);
          else
               printf(“%d”, x * 2);
           x = x + 1;

    } while x < 10;
}
```

```
#include <stdio.h>

int main()
{
    int x;
    for ( x = 0; x < 10; x++ )
    {
         for (y = 10; y > 1; y--)
           printf(“%d, %d”, x, y );
    }
}
```

Bagaimana bentuk program-program ini?



...atau yang ini?



BLACK BOX TESTING

EQUIVALENCE CLASS PARTITIONING

BOUNDARY VALUE ANALYSIS

GRAPH BASED METHOD

COMPARISON TESTING

ORTHOGONAL ARRAY TESTING

MODEL BASED TESTING

DAN LAIN-LAIN

Permasalahan Black-Box Testing

- Mencari masukan apa yang bagus untuk kasus uji
- Apakah suatu masukan data akan memungkinkan unit program yang diuji menjadi salah?
- Berapa jumlah masukan data yang perlu dicoba?
  - Mungkin perlu banyak, tetapi seberapa banyak?
- Ada nilai batas?
  - Apakah nilai-nilai batas sudah diperiksa pada kode program

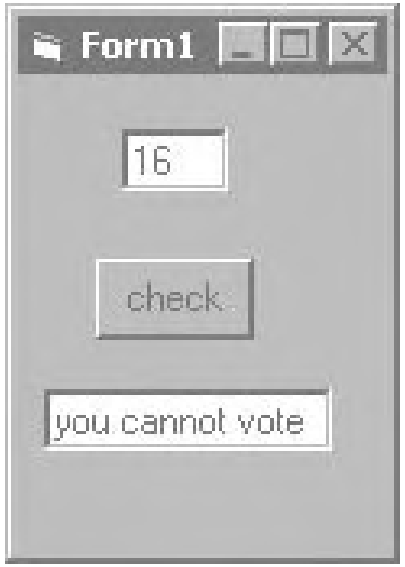
Beberapa Teknik Black Box

- **Equivalence Class Partitioning**
- **Boundary Value Analysis**
- Graph Based method
- Comparison Testing
- Orthogonal Array Testing
- Model Based Testing
- Dan lain-lain

Equivalence Class Partitioning dan Boundary Value Analysis (ECP/BVA)

Contoh

- Sebuah program digunakan untuk Pemilu. Program akan menerima data masukan umur. Umur yang dibolehkan adalah 17 tahun ke atas agar bisa memilih.
  - Ada dua kelas ekivalen (ada dua partisi)



0	16	17	Tak hingga?

– Jadi akan hanya diperlukan dua kasus uji

Nomor Test	Data	Hasil Yang Diharapkan
1	12	You cannot vote
2	21	You can vote

- Kedua test itu tentunya belum cukup, karena kita tahu bahwa test tersebut belum mencakup perbedaan batas yang penting yaitu batas umur 17 tahun. Jadi perlu diperiksa yang yang berumur 16, 17 dan 18.
  - Kenapa? Karena pemrogram bisa saja melakukan kesalahan memasukkan kondisi umur tersebut.
    - If (umur >17) atau if (umur >= 17) atau if (umur <= 16) ?

- Jadi kita buat dua test lagi menggunakan ide Boundary Value Analysis.

Nomor Test	Data	Hasil Yang Diharapkan
3	16	You cannot vote
4	17	You can vote

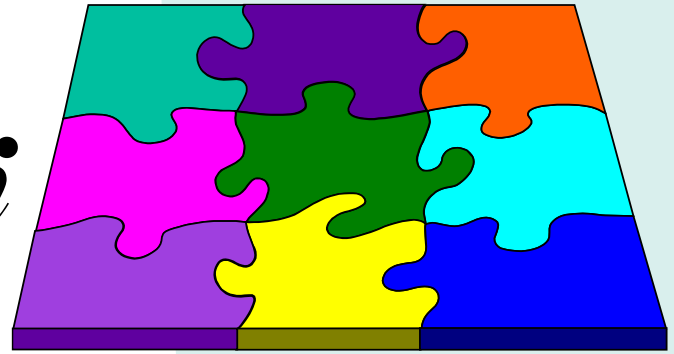
- Kenapa 18 tidak perlu diperiksa lagi?
- Apakah perlu memeriksa umur 0 dan umur 100 misalnya?

Integration Testing

Pengujian Integrasi (Integration Testing)

- Pengujian unit hanya fokus pada komponen/unit individual
  - Sesudah semua fault/bug/defect dihilangkan dari setiap unit, maka unit/komponen ini siap diintegrasikan atau digabungkan
- Proses integrasi unit-unit tadi akan memungkinkan terjadinya error yang tidak terdeteksi sebelumnya.
  - Pengujian integrasi dapat menemukan kesalahan atau bahkan dapat mendeteksi fault/bug/defect yang belum terdeteksi saat pengujian unit
  - Pengujian integrasi fokusnya lebih pada sekumpulan komponen/unit
- Idealnya, proses integrasi atau penggabungan, dilakukan satu persatu, setiap kali penggabungan satu unit maka akan dilakukan pengujian.
  - Jika kesalahan tidak ditemukan, maka unit lain akan digabungkan, hingga seluruh unit terintegrasi
  - Strategi penggabungan/integrasi akan menentukan lama tidaknya pengujian integrasi dilakukan
    - Strategi yang salah akan menambah biaya dan waktu

Strategi Pengujian Integrasi

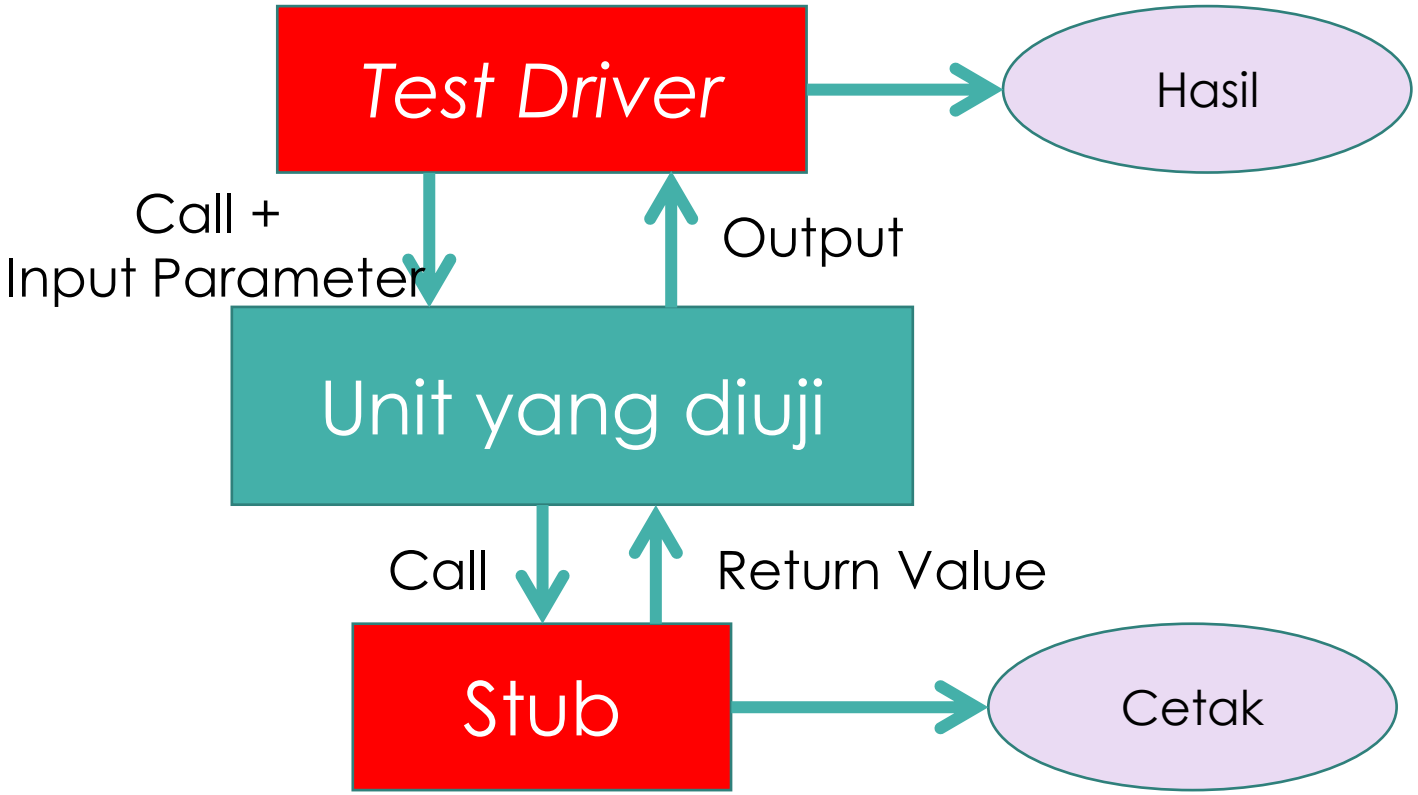


Pendekatan incremental

- Top Down testing
- Bottom Up testing
- Sandwich Testing

Pendekatan 'BIG BANG'

Review: Lingkungan Pengujian



Unit Under Test

```
int HitungGajiPegawai(int NIP)
{
    int GaPok = ReadGajiPokok(NIP);
    int GaTun = ReadGajiTun( NIP);

    return GaPok + GaTun;
```

STUB

```
int ReadGajiPokok(int NIP)
{
    // execute sql statement
    // select gaji from Pegawai
    return 2500;
}
```

```
int ReadGajiTun(int NIP)
{
    cute sql statement
    ect tunj from Pegawai
    urn 7500;
```

Test Driver

```
int main()
{
    assert(HitungGajiPegawai(111), 10000);
    assertNotTrue(HitungGajiPegawai(200), 5000);
}
```

Strategi Top Down Testing

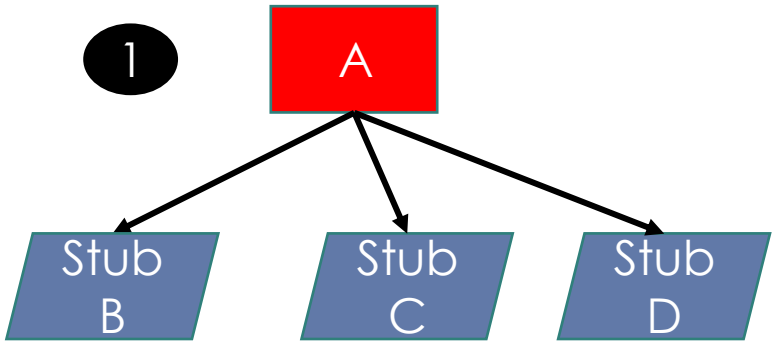
- Pengujian dimulai dari komponen level atas, biasanya bagian User Interface (UI), lalu diikuti komponen-komponen di bawahnya, hingga semua sudah diuji.
- Butuh Stub untuk komponen di bawahnya, test driver tidak diperlukan

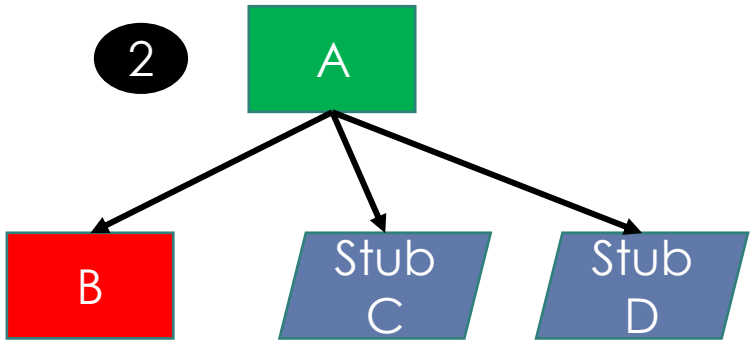
Keuntungan: Biasanya dimulai dari UI, (atau menu) atau dari program utama sehingga mudah (tidak butuh test driver)
Kerugian: kadang banyak stub dibutuhkan, padahal pengembangan stub butuh waktu dan stub sering ada error

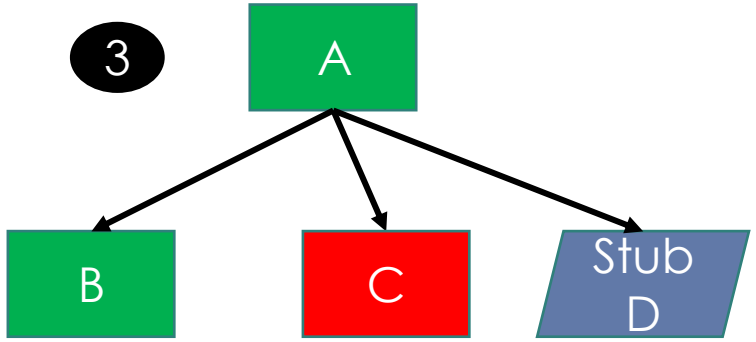
Top Down Testing

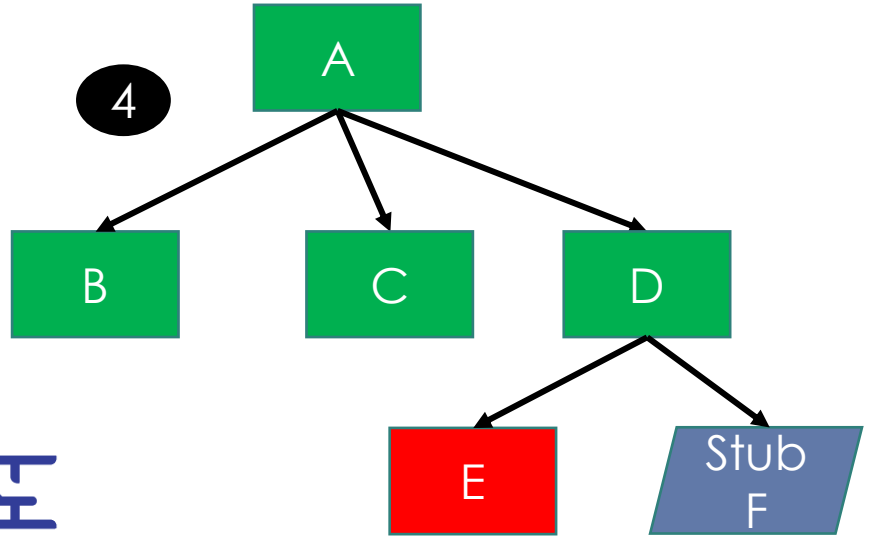
Modul yang baru digabung, perlu diuji

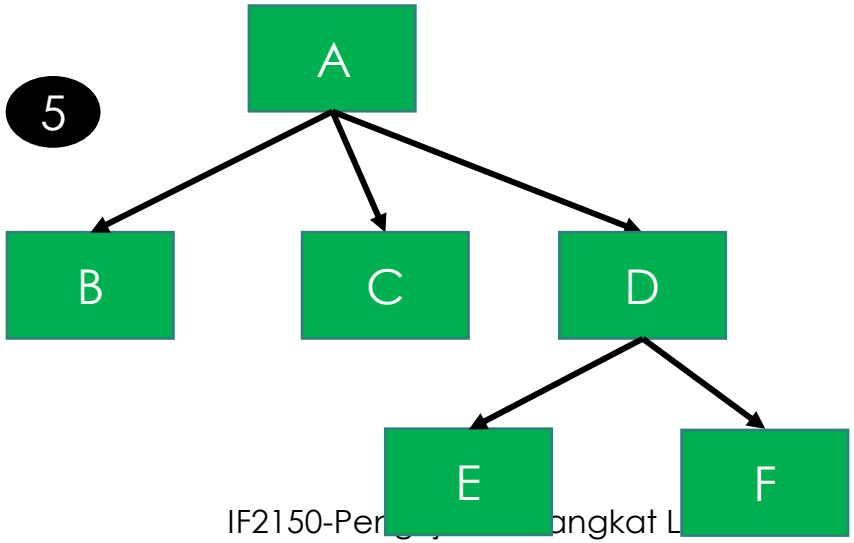
Modul yang sudah lolos uji







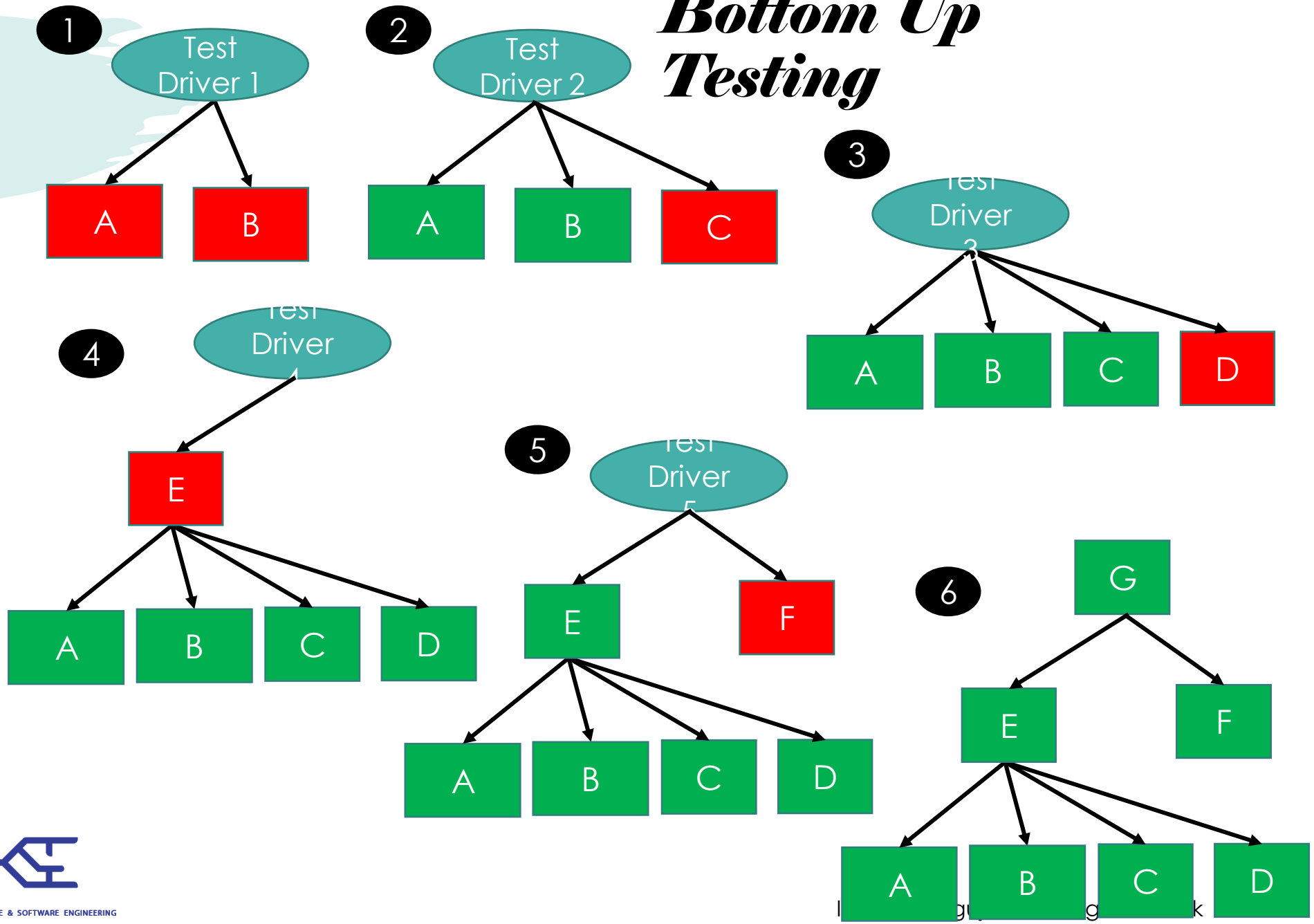




Strategi Bottom-Up Testing

- Setelah tiap komponen/unit di lapisan bawah di lakukan unit-test, maka komponen/unit ini diintegrasikan dengan komponen di lapisan atasnya.
  - Diulang hingga semua level di atasnya terintegrasi
  - Butuh test driver untuk men-simulasikan komponen/unit di level atasnya
    - Test Stub tidak diperlukan

Bottom Up Testing



Strategi Pengujian Sandwich (Sandwich Testing)

- Penggabungan strategy Top-down dan Bottom-Up
  - Sistem seolah dibagi menjadi tiga lapisan
    - Lapisan atas
    - Lapisan bawah
    - Lapisan target (yang terletak antara lapisan atas dan bawah)
  - Dengan fokus pada lapisan target, maka top-down/bottom-up testing dilakukan secara bersamaan (paralel)

Pengujian Regresi (Regression Testing)

- Pengujian Regresi melakukan re-eksekusi terhadap beberapa subset test yang sudah dilakukan sebelumnya untuk meyakinkan bahwa perubahan yang baru tidak berakibat pada hasil pengujian sebelumnya
- Jika software diperbaiki, beberapa aspek dalam konfigurasi juga perlu diperbaiki (dokumentasi, program, ataupun data)
- Pengujian regresi akan meyakinkan bahwa perubahan tidak memberikan perilaku yang tidak diinginkan atau tidak mempengaruhi modul-modul sebelumnya.
- Pengujian regresi dapat dilakukan secara manual dengan mengeksekusi ulang semua kasus uji tetapi penggunakan tools akan memungkinkan otomatisasi sehingga akan lebih cepat dan mudah
  - Misalnya penggunaan JUnit (untuk java), NUnit (.NET) atau PUnit (Php).
- Pengujian Regresi juga perlu dilakukan untuk Unit Testing

Big Bang Testing

- Pada Big Bang testing, pengujian dengan langsung menggabung semua unit/komponen tadi sebagai sistem yang utuh
  - **Keuntungan**: tidak perlu stub atau driver
  - **Kerugian**:
    - Bila terjadi suatu error mungkin sulit dicari bug-nya
      - Sulit diketahui apakah faultnya terletak di interface atau terletak di bagian dalam unit/komponenennya.
- Big Bang Testing bukanlah cara yang ideal, nampaknya cepat, tetapi untuk software skala menengah hingga besar, cara ini tidak di rekomendasikan.

Pengujian System

- Validation testing
  - Fokus pada kebutuhan perangkat lunak (software requirements)
- System testing
  - Fokus pada integrasi sistem
- Alpha/Beta testing
  - Fokus pada pengguna (bagaimana mereka menggunakan)
- Recovery testing
  - Software dibuat 'gagal' dan melihat apakah proses recovery sudah dilakukan.
    - Contoh: software yang memanfaatkan internet dilihat perilaku recovery-nya bila internet tiba-tiba putus
- Security testing
  - Memverifikasi apakah mekanisme proteksi sudah melindungi suatu 'penetrasi'
- Stress testing
  - Sistem diuji bagaimana jika menghadapi permintaan pemakaian sumberdaya yang melebihi jumlah normal, atau frekuensi atau volume normal
- Performance Testing
  - Menguji performasi software saat dieksekusi pada suatu konteks tertentu.

Pengujian Regresi (Regression Testing)

- Pengujian Regresi melakukan re-eksekusi terhadap beberapa subset test yang sudah dilakukan sebelumnya untuk meyakinkan bahwa perubahan yang baru tidak berakibat pada hasil pengujian sebelumnya
- Jika software diperbaiki, beberapa aspek dalam konfigurasi juga perlu diperbaiki (dokumentasi, program, ataupun data)
- Pengujian regresi akan meyakinkan bahwa perubahan tidak memberikan perilaku yang tidak diinginkan atau tidak mempengaruhi modul-modul sebelumnya.
- Pengujian regresi dapat dilakukan secara manual dengan mengeksekusi ulang semua kasus uji tetapi penggunakan tools diharapkan akan lebih otomatis
    - Misalnya penggunaan JUnit (untuk java), NUnit (.NET) atau PUnit (Php).

Pengujian System

- **Functional Testing:** fokus pada fungsionalitas perangkat lunak
- **Validation testing**: fokus pada kebutuhan perangkat lunak (software requirements)
- **Alpha/Beta testing**: fokus pada pengguna (bagaimana mereka menggunakan)
- **Recovery testing**: software dibuat 'gagal' dan melihat apakah proses recovery sudah dilakukan.
  - Contoh: software yang memanfaatkan internet dilihat perilaku recovery-nya bila internet tiba-tiba putus
- **Security testing**: memverifikasi apakah mekanisme proteksi sudah melindungi suatu 'penetrasi'
- **Stress testing**: sistem diuji bagaimana jika menghadapi permintaan pemakaian sumberdaya yang melebihi jumlah normal, atau frekuensi atau volume normal
- **Performance Testing**: menguji performasi software saat dieksekusi pada suatu konteks tertentu.

Pengujian Kebutuhan Fungsional

PENGUJIAN FUNGSIONAL *(FUNCTIONAL TESTING)*

Functional Testing

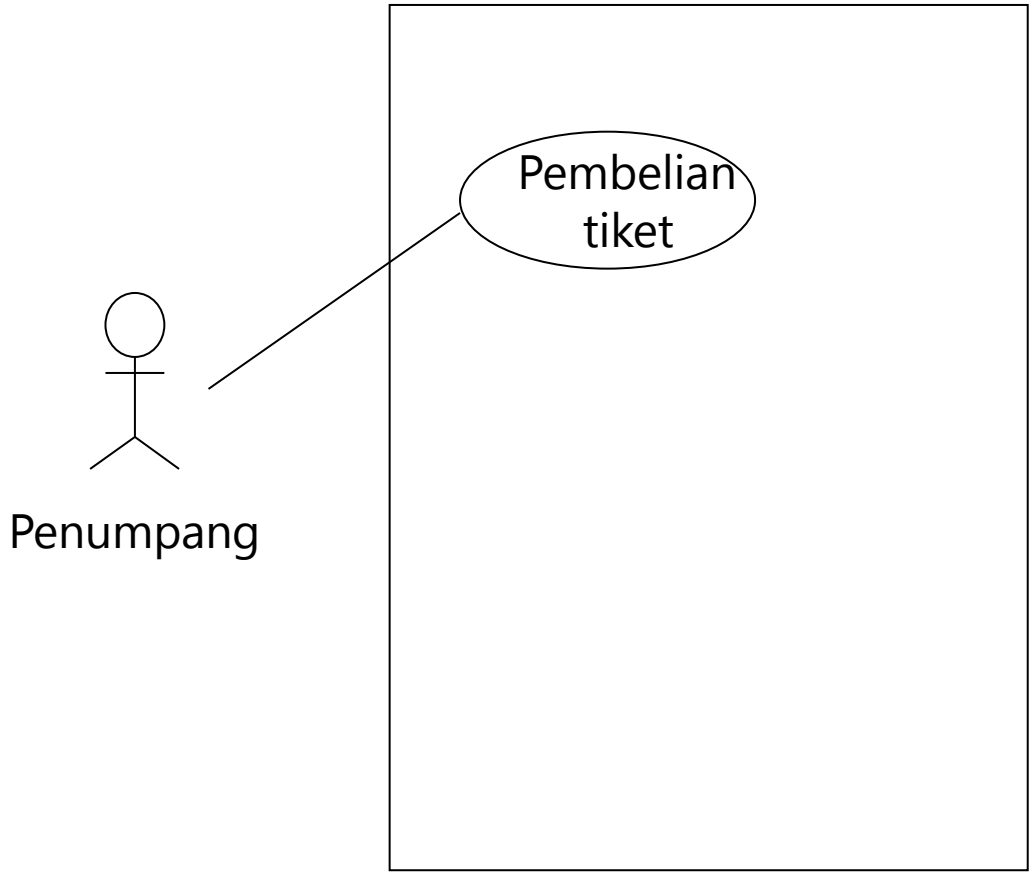
- Functional Testing adalah bagian dari System Testing
  - Contoh system testing lain: performance testing, security testing, recovery testing, acceptance testing, pilot testing dan installation testing
- Fokus dari Pengujian fungsional adalah menjamin semua kebutuhan fungsional sudah diuji
  - Hasil implementasi program harus sudah mencakup semua yang dituliskan dalam spesifikasi kebutuhan perangkat lunak
- Pengujian ini sifatnya adalah pengujian **Black-Box**
  - Memanfaatkan model use case
  - Dipilih suatu instansiasi dari use case yang kemungkinan bisa menyebabkan error
    - Bisa memanfaatkan Equivalence Class Partitioning/Boundary Testing
    - Test case dibuat berdasarkan skenario normal dan juga skenario alternatif

Studi Kasus: Mesin Tiket Kereta Api

Deskripsi Mesin Tiket Kereta Api

- Sebelum memulai perjalanan kereta api dari suatu stasiun keberangkatan, penumpang harus memilih stasiun tujuan. Mesin akan menampilkan semua tujuan yang mungkin dicapai dari stasiun keberangkatan. Setelah pemilihan tujuan, akan ditampilkan harga tiket.
  - Penumpang bisa mengubah tujuan, dan setiap kali mesin akan memberikan harga yang sesuai dengan tujuan baru.
- Sesudah memilih tujuan, penumpang dengan kategori tertentu bisa mendapatkan discount (misalnya pelajar/mahasiswa, orang tua, memiliki kebutuhan khusus)
- Kemudian penumpang dapat melakukan pembayaran dengan koin dengan nilai tertentu. Mesin akan membaca jumlah koin yang dimasukkan. Jika mencukupi, maka mesin akan mengeluarkan tiket dan mengembalikan uang jika diperlukan.

Contoh use case pembelian karcis kereta lewat mesin tiket



Diketahui ada 5 stasiun tujuan, dengan biaya masing-masing

- stasiun 1: 1000,
- stasiun 2: 1500 ,
- stasiun 3: 2500,
- stasiun 4: 3000,
- stasiun 5: 4000

Use case: Pembelian Tiket

Kondisi Awal: Penumpang berdiri di depan mesin tiket dengan uang yang cukup

Skenario Normal:

- Penumpang memilih tujuan stasiun, dengan menekan tombol tujuan, bila ada lebih dari satu yang ditekan, maka pilihan terakhir yang diambil
- Mesin akan menampilkan biaya yang harus dibayar
- Penumpang memasukkan koin ke slot mesin
  - Alternatif skenario
    - Jika penumpang memasukkan stasiun baru sebelum uang yang dimasukkan cukup, maka mesin akan mengembalikan seluruh koin
    - Jika penumpang memasukkan jumlah koin lebih, maka pengembalian akan diberikan oleh mesin
- Mesin akan mencetak tiket
- Penumpang akan mengambil tiket dan kembalian jika ada

Kondisi akhir: Tiket sudah ditangan penumpang

Ide Pengujian

- Dari use-case tersebut, maka ada beberapa fitur dari mesin tiket tersebut yang harus diuji
  - Jika penumpang menekan tombol dan memasukkan jumlah koin yang tepat, maka ia akan mendapatkan tiket saja.
  - Jika penumpang menekan tombol untuk beberapa stasiun, maka mesin harus menampilkan harga tiket stasiun terakhir
  - Jika penumpang menekan tombol stasiun lain, selagi memasukkan uang ke mesin, maka uang otomatis dikembalikan dulu
  - Jika penumpang memasukkan uang yang melebihi yang harus dibayar, mesin harus melakukan pengembalian

Test Case 1

Nama Test case	TC1: Uang Pas
Kondisi awal	Penumpang di depan mesin, dan memiliki koin 5 koin 1000, 2 koin 500
Skenario	1. Penumpang menekan tombol stasiun 3 dan mesin akan menampilkan 2500 2. Penumpang memasukkan dua koin 1000 dan satu koin 500
Kondisi akhir	Penumpang mendapatkan tiket untuk stasiun 3 dan tidak ada koin kembalian

Test Case 2

Nama Test case	TC2: Penekanan tombol berbeda
Kondisi awal	Penumpang di depan mesin, dan memiliki koin 5 koin 1000, 2 koin 500
Skenario	1. Penumpang menekan tombol stasiun 5, 3, 1, 2, pada saat yang bersamaan setiap kali mesin akan menampilkan 4000, 2500, 1000, lalu 1500 2. Penumpang memasukkan dua koin 1000 3. Mesin harus mengembalikan koin sejumlah 500
Kondisi akhir	Penumpang mendapat tiket ke stasiun 2 dan mendapatkan pengambalian koin 500

Test Case 3

Nama Test case	TC3: pemilihan stasiun lain saat koin sudah dimasukkan
Kondisi awal	Penumpang di depan mesin, dan memiliki koin 5 koin 1000, 2 koin 500
Skenario	1. Penumpang menekan tombol stasiun 5 dan mesin akan menampilkan 4000 2. Penumpang memasukkan dua koin 1000 3. Lalu penumpang menekan tombol 3 4. Mesin harus langsung mengembalikan koin sejumlah 2000
Kondisi akhir	Penumpang mendapat Koin kembali sejumlah 2000

Test Case 4

Nama Test case	TC4: Pengembalian Koin
Kondisi awal	Penumpang di depan mesin, dan memiliki koin 5 koin 1000, 2 koin 500
Skenario	1. Penumpang menekan tombol stasiun 3 dan mesin akan menampilkan 2500 2. Penumpang memasukkan tiga koin 1000 3. Mesin harus mengembalikan koin sejumlah 500
Kondisi akhir	Penumpang mendapatkan tiket dan koin kembali sejumlah 500